

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN



ĐỒ ÁN 02
XỬ LÝ ẢNH (IMAGE PROCESSING)

MÔN: TOÁN ỨNG DỤNG CHO CÔNG NGHỆ THÔNG TIN

Giảng viên hướng dẫn:

Nguyễn Đình Thúc
Nguyễn Văn Quang Huy
Ngô Đình Hy

Sinh viên thực hiện:

MSSV – 21127235
Nguyễn Xuân Quỳnh Chi

Thành phố Hồ Chí Minh, tháng 8 năm 2023

MỤC LỤC

<i>Thông tin sinh viên</i>	<i>3</i>
<i>Bảng liệt kê mức độ hoàn thành các chức năng.....</i>	<i>3</i>
<i>Ý tưởng thực hiện và mô tả các hàm chức năng.....</i>	<i>3</i>
1. Điều chỉnh độ sáng của ảnh.....	3
2. Điều chỉnh độ tương phản của ảnh	4
3. Lật ảnh ngang/dọc.....	4
4. Chuyển đổi ảnh RGB thành ảnh xám	5
5. Làm mờ/sắc nét ảnh.....	6
6. Cắt ảnh theo khung tròn	7
<i>Tài liệu tham khảo</i>	<i>8</i>

Thông tin sinh viên

MSSV: 21127235

Họ tên: Nguyễn Xuân Quỳnh Chi

Lớp: 21CLC04

Bảng liệt kê mức độ hoàn thành các chức năng

STT	Chức năng	Mức độ hoàn thành
1	Điều chỉnh độ sáng	100%
2	Điều chỉnh độ tương phản	100%
3	Lật ảnh ngang/đọc	100%
4	Chuyển đổi ảnh RGB thành ảnh xám	100%
5	Làm mờ/sắc nét ảnh	100%
6	Cắt ảnh theo kích thước (cắt ở trung tâm)	0%
7	Cắt ảnh theo khung tròn	100%

Ý tưởng thực hiện và mô tả các hàm chức năng

1. Điều chỉnh độ sáng của ảnh

1.1. Ý tưởng

- Cộng giá trị brightness vào từng kênh màu R, G, B của từng pixel.
- Nếu brightness càng lớn thì ảnh càng sáng, ngược lại nếu brightness càng nhỏ thì ảnh càng tối. Do đó, nếu brightness quá lớn hoặc quá nhỏ, tức là ngoài khoảng $[0, 255]$ thì ảnh hoặc sẽ bị cháy trắng hoặc sẽ bị tối đen. Để ảnh không bị rơi vào hai trường hợp này, khi cộng brightness vào từng kênh màu của các pixel, nếu giá trị kênh màu nào vượt quá 255 thì lấy 255, hoặc nếu giá trị kênh màu nào dưới 0 thì lấy 0.

1.2. Input: Tập tin ảnh và độ sáng cần điều chỉnh.

Output: Ảnh sau khi đã thay đổi độ sáng.

1.3. Mô tả

- Cộng mảng image với giá trị brightness được chuyển đổi về dạng float để đảm bảo tính chính xác.
- Giới hạn miền giá trị cho các phần tử của ảnh là $[0, 255]$ vì các giá trị kênh màu của mỗi pixel chỉ nằm trong khoảng $[0, 255]$.

1.4. Hình ảnh



Hình 1: Ảnh được chỉnh độ sáng brightness = 92

2. Điều chỉnh độ tương phản của ảnh

2.1. Ý tưởng

- Lần lượt thay đổi giá trị các kênh màu R, G, B của từng pixel theo công thức (theo kênh màu R, tương tự với các kênh màu còn lại): $R' = F * R - 128 * F + 128$.
- Trong đó,

$$F = \frac{259 * (255 + \alpha)}{255 * (259 - \alpha)}$$

- F cần được chuyển về dạng float để tăng độ chính xác cho quá trình xử lý ảnh.
- Nếu giá trị contrast càng lớn thì ảnh sẽ có độ tương phản càng cao và ngược lại.
- Vì các giá trị kênh màu của mỗi pixel chỉ nằm trong khoảng [0, 255] nên nếu giá trị kênh màu nào vượt quá 255 thì lấy 255, hoặc dưới 0 thì lấy 0.

2.2. Input: Tập tin ảnh cần và độ tương phản cần điều chỉnh.

Output: Ảnh sau khi đã điều chỉnh độ tương phản.

2.3. Mô tả

- Tính giá trị F theo công thức.
- Nhân các giá trị kênh màu của từng pixel với F theo công thức.
- Giới hạn miền giá trị cho các phần tử của ảnh là [0, 255] vì các giá trị kênh màu của mỗi pixel chỉ nằm trong khoảng [0, 255].

2.4. Hình ảnh



Hình 2: Ảnh được chỉnh độ tương phản contrast = 117

3. Lật ảnh ngang/dọc

3.1. Ý tưởng

- 3.1.1. Lật ảnh ngang: Hoán đổi vị trí của các cột trong ma trận pixel: cột 0 hoán đổi vị trí với cột ($\text{width_of_matrix} - 1$); cột 1 hoán đổi vị trí với cột ($\text{width_of_matrix} - 2$); tổng quát ta có cột i hoán đổi vị trí với cột ($\text{width_of_matrix} - 1 - i$).
- 3.1.2. Lật ảnh dọc: Hoán đổi vị trí của các dòng trong ma trận pixel: dòng 0 hoán đổi vị trí với dòng ($\text{height_of_matrix} - 1$); dòng 1 hoán đổi vị trí với dòng ($\text{height_of_matrix} - 2$); tổng quát ta có dòng i hoán đổi vị trí với dòng ($\text{height_of_matrix} - 1 - i$).
- 3.2. Input: Tập tin ảnh và hướng lật ảnh (ngang, dọc).
Output: Ảnh đã được lật theo hướng yêu cầu.
- 3.3. Mô tả
- Sử dụng các hàm có sẵn trong thư viện NumPy để lật ảnh theo hướng:
 - + `flipud(image)` để lật ảnh theo chiều dọc;
 - + `fliplr(image)` để lật ảnh theo chiều ngang.
- 3.4. Hình ảnh



Hình 3: Ảnh được lật dọc



Hình 4: Ảnh được lật ngang

4. Chuyển đổi ảnh RGB thành ảnh xám

4.1. Ý tưởng

- Với mỗi pixel, gộp ba kênh màu R, G, B thành một kênh màu duy nhất theo công thức:

$$\text{Grayscale pixel} = (0.3 * R) + (0.59 * G) + (0.11 * B)$$

- Mỗi kênh màu sẽ có một trọng số riêng: R - 0.3, G - 0.59 và B - 0.11.
- 4.2. Input: Tập tin ảnh cần chuyển về ảnh xám.
Output: Ảnh xám của ảnh gốc ban đầu.

4.3. Mô tả

- Khởi tạo mảng trọng số weight [0.3, 0.59, 0.11]

- Nhân vector trọng số này với ma trận ảnh.

4.4. Hình ảnh



Hình 5: Ảnh được chỉnh về ảnh xám

5. Làm mờ/sắc nét ảnh

- *Làm mờ ảnh*

5.1. Ý tưởng

- Sử dụng một ma trận kernel kiểu Gaussian blur 3x3 để làm mờ ảnh bằng phép tính convolution: Ứng với mỗi phần tử trong ma trận ảnh ban đầu (xij), ta lấy ra một ma trận có kích thước đúng bằng kích thước của ma trận kernel 3x3, tạm gọi là ma trận A. Sau đó lần lượt tính tổng các phần tử của phép tính element-wise của ma trận A với ma trận kernel. (Element-wise multiplication matrix: Mỗi phần tử của ma trận kết quả bằng tích hai phần tử tương ứng ở hai ma trận ban đầu).
- Như vậy, với mỗi lần thực hiện phép tính convolution trên ma trận ảnh, ta luôn được ma trận kết quả có kích thước nhỏ hơn. Để tạo ra ma trận kết quả có cùng kích thước với ma trận ảnh ban đầu, ta thêm padding = k, tức là thêm k vector 0 vào mỗi phía của ma trận ảnh ban đầu (viên ngoài).

5.2. Input: Tập tin ảnh cần được làm mờ.

Output: Tập tin ảnh đã được làm mờ.

5.3. Mô tả

- Khởi tạo ma trận kernel kiểu Gaussian blur 3x3:

$$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

- Tạo một ma trận ảnh với toàn các pixel [0, 0, 0] có kích thước đúng bằng kích thước của ma trận ảnh ban đầu sau khi thêm vào padding, gọi là padding_matrix. Sau đó, thay phần trung tâm của ma trận padding_matrix bằng ma trận ảnh ban đầu cần được làm mờ.
- Dùng hai vòng lặp for để thực hiện phép tính convolution trên ma trận ảnh: padding_matrix[col:col+3, row:row+3] * kernel: thực hiện phép tính element-wise đối với ma trận kernel và ma trận 3x3 có phần tử trung tâm (row, col) chính là pixel ta muốn làm mờ.

- *Làm sắc nét ảnh*
- Ý tưởng và cài đặt tương tự với kỹ thuật làm mờ ảnh.
- Điểm khác biệt: Sử dụng một ma trận kernel khác:

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

5.4. Hình ảnh



Hình 6: Ảnh được làm mờ

6. Cắt ảnh theo khung tròn

6.1. Ý tưởng

- Tạo một mặt nạ (mask) có dạng hình tròn có tâm nằm ngay chính giữa khung hình, sau đó phủ lên ảnh gốc ban đầu.

6.2. Input: Chiều cao và chiều rộng của ảnh.

Output: Ảnh sau khi được cắt theo khung tròn.

6.3. Mô tả

- Khởi tạo toạ độ tâm của khung tròn trùng với vị trí trung tâm của khung hình.
- Bán kính của khung tròn là khoảng cách ngắn nhất từ vị trí tâm đến các cạnh của khung hình.
- Dùng hàm np.ogrid để tạo giá trị toạ độ y, x tương ứng với chiều dài và chiều rộng của khung hình.
- Tạo mask có dạng hình tròn với x, y là các điểm thuộc đường tròn, áp dụng bất phương trình đường tròn:

$$(x - a)^2 + (y - b)^2 > R^2$$

R là bán kính đường tròn tâm $I(a, b)$

6.4. Hình ảnh



Hình 7: Ảnh được cắt theo khung tròn

Tài liệu tham khảo

1. [https://en.wikipedia.org/wiki/Kernel_\(image_processing\)](https://en.wikipedia.org/wiki/Kernel_(image_processing))
2. <https://www.dfstudios.co.uk/articles/programming/image-programming-algorithms/image-processing-algorithms-part-4-brightness-adjustment/>
3. <https://github.com/kieuconghau/image-processing/blob/master/Source/18127259.ipynb>
4. <https://github.com/ThongLai/ImageProcessing/blob/main/20127635.ipynb>
5. <https://numpy.org/doc/stable/reference/generated/numpy.fliplr.html>
6. <https://numpy.org/doc/stable/reference/generated/numpy.flipud.html>
7. https://www.tutorialspoint.com/dip/grayscale_to_rgb_conversion.htm
8. <https://www.dfstudios.co.uk/articles/programming/image-programming-algorithms/image-processing-algorithms-part-5-contrast-adjustment/>
9. https://en.wikipedia.org/wiki/Gaussian_blur